Delegate - C++14

Generated by Doxygen 1.16.1

1 Namespace Index 1
1.1 Namespace List . . . 1

2 Hierarchical Index 3
2.1 Class Hierarchy . . . 3

3 Class Index 5
3.1 Class List . . . 5

4 File Index 7
4.1 File List . . . 7

5 Namespace Documentation 9
5.1 bricxx Namespace Reference . . . 9

6 Class Documentation 11
6.1 bricxx::BadDelegateCall Class Reference . . . 11
6.1.1 Detailed Description . . . 11
6.1.2 Member Function Documentation . . . 11
6.1.2.1 what() . . . 11
6.2 bricxx::Delegate< TSignature > Class Template Reference . . . 12
6.2.1 Detailed Description . . . 12
6.3 bricxx::Delegate< TRetVal(TParams...) const > Class Template Reference . . . 13
6.3.1 Detailed Description . . . 14
6.3.2 Constructor & Destructor Documentation . . . 15
6.3.2.1 Delegate() [1/2] . . . 15
6.3.2.2 Delegate() [2/2] . . . 15
6.3.2.3 ~Delegate() . . . 15
6.3.3 Member Function Documentation . . . 15
6.3.3.1 operator=() . . . 15
6.3.3.2 operator==() . . . 15
6.3.3.3 operator"!=() . . . 15
6.3.3.4 Bind() [1/2] . . . 16
6.3.3.5 Bind() [2/2] . . . 16
6.3.3.6 IsBindedTo() [1/2] . . . 16
6.3.3.7 IsBindedTo() [2/2] . . . 16
6.3.3.8 operator()() . . . 16
6.4 bricxx::Delegate< TRetVal(TParams...)> Class Template Reference . . . 17
6.4.1 Detailed Description . . . 18
6.4.2 Constructor & Destructor Documentation . . . 18
6.4.2.1 Delegate() [1/2] . . . 18
6.4.2.2 Delegate() [2/2] . . . 18
6.4.2.3 ~Delegate() . . . 18
6.4.3 Member Function Documentation . . . 18

6.4.3.1 operator=() . . . 18
6.4.3.2 operator==() . . . 18
6.4.3.3 operator"!=() . . . 19
6.4.3.4 Bind() [1/3] . . . 19
6.4.3.5 Bind() [2/3] . . . 19
6.4.3.6 Bind() [3/3] . . . 19
6.4.3.7 IsBindedTo() [1/3] . . . 19
6.4.3.8 IsBindedTo() [2/3] . . . 19
6.4.3.9 IsBindedTo() [3/3] . . . 20
6.4.3.10 operator()() . . . 20
6.5 bricxx::DelegateBase< TRetVal, TParams > Class Template Reference . . . 20
6.5.1 Detailed Description . . . 21
6.5.2 Member Typedef Documentation . . . 21
6.5.2.1 ErasedObjectType . . . 21
6.5.2.2 StubFnPtr . . . 22
6.5.3 Constructor & Destructor Documentation . . . 22
6.5.3.1 DelegateBase() [1/2] . . . 22
6.5.3.2 DelegateBase() [2/2] . . . 22
6.5.3.3 ~DelegateBase() . . . 22
6.5.4 Member Function Documentation . . . 22
6.5.4.1 IsBinded() . . . 22
6.5.4.2 operator bool() . . . 22
6.5.4.3 Unbind() . . . 23
6.5.4.4 Reset() . . . 23
6.5.4.5 operator=() . . . 23
6.5.4.6 operator==() . . . 23
6.5.4.7 operator"!=() . . . 23
6.5.4.8 NullStub() . . . 23
6.5.5 Member Data Documentation . . . 24
6.5.5.1 pStub_ . . . 24
6.5.5.2 pTypeErasedObject_ . . . 24

7 File Documentation **25**
7.1 delegate.hpp File Reference . . . 25
7.1.1 Macro Definition Documentation . . . 25
7.1.1.1 BRICXX_DELEGATE_API . . . 25
7.2 delegate.hpp . . . 26

Index **33**

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

bricxx 9

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

bricxx::Delegate< TSignature > 12
bricxx::DelegateBase< TRetVal, TParams > 20
bricxx::DelegateBase< TRetVal, TParams... > 20
  bricxx::Delegate< TRetVal(TParams...) const > 13
  bricxx::Delegate< TRetVal(TParams...)> 17
std::exception
  bricxx::BadDelegateCall 11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

bricxx::BadDelegateCall
Exception thrown by `Delegate::operator()()` if the delegate is not binded 11

bricxx::Delegate< TSignature >
A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined 12

bricxx::Delegate< TRetVal(TParams...) const >
`Delegate` partial specialization for const 13

bricxx::Delegate< TRetVal(TParams...)>
`Delegate` partial specialization for non-const 17

bricxx::DelegateBase< TRetVal, TParams >
Base class for `Delegate` partial specializations 20

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

delegate.hpp 25

Chapter 5

Namespace Documentation

5.1 bricxx Namespace Reference

Classes

- class Delegate

  A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined.
- class BadDelegateCall

  Exception thrown by `Delegate::operator()()` if the delegate is not binded.
- class DelegateBase

  Base class for `Delegate` partial specializations.
- class Delegate< TRetVal(TParams...)>

  `Delegate` partial specialization for non-const.
- class Delegate< TRetVal(TParams...) const >

  `Delegate` partial specialization for const.

Chapter 6

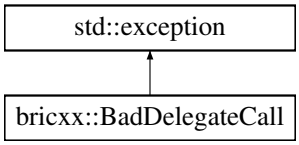
Class Documentation

6.1 bricxx::BadDelegateCall Class Reference

Exception thrown by `Delegate::operator()()` if the delegate is not binded.

`#include <delegate.hpp>`

Inheritance diagram for bricxx::BadDelegateCall:



Public Member Functions

- const char * what () const noexcept override

6.1.1 Detailed Description

Exception thrown by `Delegate::operator()()` if the delegate is not binded.

Definition at line 157 of file delegate.hpp.

6.1.2 Member Function Documentation

6.1.2.1 what()

```
const char * bricxx::BadDelegateCall::what () const  [override], [noexcept]
```

The documentation for this class was generated from the following file:

- delegate.hpp

6.2 bricxx::Delegate< TSignature > Class Template Reference

A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined.

6.2.1 Detailed Description

template<typename TSignature>
class bricxx::Delegate< TSignature >

A non-owning, const-correct and equality comparable callable wrapper. Primary template, not defined.

`Delegate`:

- is copyable, small (two pointers) and fast (no dynamic memory allocation, no virtual functions, all functions are inline (except stubs because their address is needed), delegate call involves only two extra calls (operator()() and stub function)).
- supports (non-member) functions, methods (member functions) and functors/lambda.
- is a single header library so it is easy to integrate in any project.

Attention

Because `Delegate` is non-owning special care is needed to ensure that the binded target outlive it.

Remarks

- Two delegates are considered equal if they are not binded or binded to the same target.
- Equality comparison and `Delegate::IsBindedTo()` allows delegates to be easily used as callbacks that can be registered/unregistered (`::std::function_ref` is not equality comparable).
- The solutions presented in **references** directory are not const-correct (they have the same problem as `::std::function`).

See also

- `Rich Hickey: Callbacks in C++ Using Template Functors`
- Bert Peers: Callbacks in C++
- Don Clugston, CodeProject: Member Function Pointers and the Fastest Possible C++ Delegates
- Sergey Ryazanov, CodeProject: The Impossibly Fast C++ Delegates
- Sergey Alexandrovich Kryukov, CodeProject: The Impossibly Fast C++ Delegates, Fixed
- Aleksei Trunov, CodeProject: Yet Another Generalized Functors Implementation in C++
- JaeWook Choi, CodeProject: Fast C++ Delegate
- `Matthew Rodusek: Creating a Fast and Efficient Delegate Type (Part 1)`
- `Matthew Rodusek: Creating a Fast and Efficient Delegate Type (Part 2)`
- `Matthew Rodusek: Creating a Fast and Efficient Delegate Type (Part 3)`
- `Matthew Rodusek: Unbelievably Fast Delegate`

- user1095108: Impossibly fast delegate in C++11
- user1095108: delegate.hpp
- **references** directory in this project that contains the above.

Minimal introduction/examples (please see tests for more details):

```
// (non-member) function
Delegate<int (int)> functionDelegate;
functionDelegate.Bind<&::islower>();
if (!functionDelegate.IsBindedTo<&::islower>()) throw runtime_error("function test failed");
if (!functionDelegate('a')) throw runtime_error("function test failed");

// lambda/functor
int numCalls = 0;
auto lambda = [&numCalls](int x, int y) mutable -> bool {
    ++numCalls;
    return x > y;
};
Delegate<bool (int, int)> lambdaDelegate;
lambdaDelegate.Bind(lambda);
if (!lambdaDelegate.IsBindedTo(lambda)) throw runtime_error("lambda test failed");
if (!lambdaDelegate(2, 1)) throw runtime_error("lambda test failed");
if (numCalls != 1) throw runtime_error("lambda test failed");

// method (member function)
string hello = "Hello, World!";
Delegate<void ()> methodDelegate;
methodDelegate.Bind<string, &string::clear>(hello);
methodDelegate();
if (!hello.empty()) throw runtime_error("method test failed");

// comparison
Delegate<int (int)> functionDelegate1;
functionDelegate1.Bind<&::isupper>();
if (functionDelegate == functionDelegate1) throw runtime_error("comparison test failed");
functionDelegate1.Bind<&::islower>();
if (functionDelegate != functionDelegate1) throw runtime_error("comparison test failed");

// const-correctness
Delegate<void () const> constDelegate;
//constDelegate.Bind<string, &string::clear>(hello);    // OK, does not compile
(void)constDelegate;

// unbind
functionDelegate.Unbind();
if (functionDelegate.IsBinded()) throw runtime_error("unbind test failed");

// BadDelegateCall
bool caught = false;
try {
    functionDelegate('a');
}
catch (const BadDelegateCall&) {
    caught = true;
}
if (!caught) throw runtime_error("BadDelegateCall test failed");
```

Definition at line 150 of file delegate.hpp.

The documentation for this class was generated from the following file:

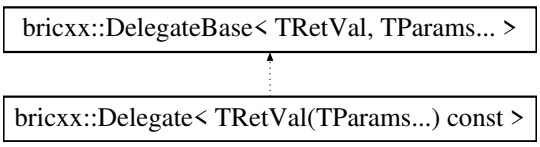
- delegate.hpp

6.3 bricxx::Delegate< TRetVal(TParams...) const > Class Template Reference

`Delegate` partial specialization for const.

`#include <delegate.hpp>`

Inheritance diagram for bricxx::Delegate< TRetVal(TParams...) const >:



Public Member Functions

- ~Delegate () noexcept
- constexpr Delegate & operator= (const Delegate &other) noexcept
- template<typename... UParams>
  TRetVal operator() (UParams &&... params) const

Constructors.

- constexpr Delegate () noexcept
  Creates a delegate that is not binded.
- constexpr Delegate (const Delegate &other) noexcept

Equality operators.

- constexpr bool operator== (const Delegate &other) const noexcept
- constexpr bool operator!= (const Delegate &other) const noexcept

Binders.

Binds the delegate to a target.

- template<class TFunctor, typename = typename ::std::enable_if< ::std::is_class<TFunctor>::value >::type>
  void Bind (const TFunctor &functor) noexcept
  Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).
- template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
  void Bind (const TClass &object) noexcept
  Binds the delegate to an `object` + method (member function).

Binder target testers (convenience methods).

Test if the delegate is binded to the given target.

- template<class TFunctor, typename = typename ::std::enable_if< ::std::is_class<TFunctor>::value >::type>
  bool IsBindedTo (const TFunctor &functor) const noexcept
  Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).
- template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
  bool IsBindedTo (const TClass &object) const noexcept
  Test if the delegate is binded to the given `object` + method (member function).

6.3.1 Detailed Description

template<typename TRetVal, typename... TParams>
class bricxx::Delegate< TRetVal(TParams...) const >

`Delegate` partial specialization for const.

Definition at line 313 of file delegate.hpp.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 Delegate() [1/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const >::Delegate () [constexpr], [noexcept]
```

Creates a delegate that is not binded.

6.3.2.2 Delegate() [2/2]

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const >::Delegate (
            const Delegate< TRetVal(TParams...) const > & other) [constexpr], [noexcept]
```

6.3.2.3 ~Delegate()

```
template<typename TRetVal, typename... TParams>
bricxx::Delegate< TRetVal(TParams...) const >::~Delegate () [noexcept]
```

Remarks

Not virtual.

6.3.3 Member Function Documentation

6.3.3.1 operator=()

```
template<typename TRetVal, typename... TParams>
Delegate & bricxx::Delegate< TRetVal(TParams...) const >::operator= (
            const Delegate< TRetVal(TParams...) const > & other) [constexpr], [noexcept]
```

6.3.3.2 operator==()

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...) const >::operator== (
            const Delegate< TRetVal(TParams...) const > & other) const [constexpr], [noexcept]
```

6.3.3.3 operator"!=()

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...) const >::operator!= (
            const Delegate< TRetVal(TParams...) const > & other) const [constexpr], [noexcept]
```

6.3.3.4 Bind() [1/2]

```
template<typename TRetVal, typename... TParams>
template<class TFunctor, typename = typename ::std::enable_if< ::std::is_class<TFunctor>::↩
value >::type>
void bricxx::Delegate< TRetVal(TParams...) const >::Bind (
            const TFunctor & functor) [noexcept]
```

Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).

6.3.3.5 Bind() [2/2]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
void bricxx::Delegate< TRetVal(TParams...) const >::Bind (
            const TClass & object) [noexcept]
```

Binds the delegate to an `object` + method (member function).

6.3.3.6 IsBindedTo() [1/2]

```
template<typename TRetVal, typename... TParams>
template<class TFunctor, typename = typename ::std::enable_if< ::std::is_class<TFunctor>::↩
value >::type>
bool bricxx::Delegate< TRetVal(TParams...) const >::IsBindedTo (
            const TFunctor & functor) const [noexcept]
```

Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).

6.3.3.7 IsBindedTo() [2/2]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) const TMethod>
bool bricxx::Delegate< TRetVal(TParams...) const >::IsBindedTo (
            const TClass & object) const [noexcept]
```

Test if the delegate is binded to the given `object` + method (member function).

6.3.3.8 operator()()

```
template<typename TRetVal, typename... TParams>
template<typename... UParams>
TRetVal bricxx::Delegate< TRetVal(TParams...) const >::operator() (
            UParams &&... params) const
```

Invokes the stub function with the given parameters.

Remarks

Perfect forwarding and converting parameter pack to stub function (`UParams` is a deduced type, `TParams` is not).

Exceptions

BadDelegateCall	if the delegate is not binded.

The documentation for this class was generated from the following file:

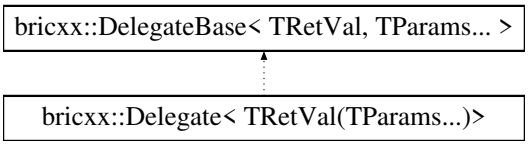
- delegate.hpp

6.4 bricxx::Delegate< TRetVal(TParams...)> Class Template Reference

`Delegate` partial specialization for non-const.

`#include <delegate.hpp>`

Inheritance diagram for bricxx::Delegate< TRetVal(TParams...)>:



Public Member Functions

- ~Delegate () noexcept
- constexpr Delegate & operator= (const Delegate &other) noexcept
- template<typename... UParams>
  TRetVal operator() (UParams &&... params)

Constructors.

- constexpr Delegate () noexcept
  Creates a delegate that is not binded.
- constexpr Delegate (const Delegate &other) noexcept

Equality operators.

- constexpr bool operator== (const Delegate &other) const noexcept
- constexpr bool operator!= (const Delegate &other) const noexcept

Binders.

Binds the delegate to a target.

- template<TRetVal(*)(TParams...) TFunction>
  constexpr void Bind () noexcept
  Binds the delegate to a (non-member) function.
- template<class TFunctor, typename = typename ::std::enable_if< ::std::is_class<TFunctor>::value >::type>
  void Bind (TFunctor &functor) noexcept
  Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).
- template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
  void Bind (TClass &object) noexcept
  Binds the delegate to an `object` + method (member function).

Binder target testers (convenience methods).

Test if the delegate is binded to the given target.

- template<TRetVal(*)(TParams...) TFunction>
  constexpr bool IsBindedTo () const noexcept
  Test if the delegate is binded to the given (non-member) function.
- template<class TFunctor, typename = typename ::std::enable_if< ::std::is_class<TFunctor>::value >::type>
  bool IsBindedTo (TFunctor &functor) const noexcept
  Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).
- template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
  bool IsBindedTo (TClass &object) const noexcept
  Test if the delegate is binded to the given `object` + method (member function).

6.4.1 Detailed Description

template<typename TRetVal, typename... TParams>
class bricxx::Delegate< TRetVal(TParams...)>

`Delegate` partial specialization for non-const.

Definition at line 212 of file delegate.hpp.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 Delegate() [1/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)>::Delegate ()  [constexpr], [noexcept]
```

Creates a delegate that is not binded.

6.4.2.2 Delegate() [2/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)>::Delegate (
            const Delegate< TRetVal(TParams...)> & other)  [constexpr], [noexcept]
```

6.4.2.3 ~Delegate()

```
template<typename TRetVal, typename...  TParams>
bricxx::Delegate< TRetVal(TParams...)>::~Delegate ()  [noexcept]
```

Remarks

Not virtual.

6.4.3 Member Function Documentation

6.4.3.1 operator=()

```
template<typename TRetVal, typename...  TParams>
Delegate & bricxx::Delegate< TRetVal(TParams...)>::operator= (
            const Delegate< TRetVal(TParams...)> & other)  [constexpr], [noexcept]
```

6.4.3.2 operator==()

```
template<typename TRetVal, typename...  TParams>
bool bricxx::Delegate< TRetVal(TParams...)>::operator== (
            const Delegate< TRetVal(TParams...)> & other) const  [constexpr], [noexcept]
```

6.4.3.3 operator"!=()

```
template<typename TRetVal, typename... TParams>
bool bricxx::Delegate< TRetVal(TParams...)>::operator!= (
            const Delegate< TRetVal(TParams...)> & other) const [constexpr], [noexcept]
```

6.4.3.4 Bind() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*)(TParams...) TFunction>
void bricxx::Delegate< TRetVal(TParams...)>::Bind () [constexpr], [noexcept]
```

Binds the delegate to a (non-member) function.

6.4.3.5 Bind() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFunctor, typename = typename ::std::enable_if< ::std::is_class<TFunctor>::↩
value >::type>
void bricxx::Delegate< TRetVal(TParams...)>::Bind (
            TFunctor & functor) [noexcept]
```

Binds the delegate to a `functor` / lambda (`TFunctor` is auto-deduced).

6.4.3.6 Bind() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
void bricxx::Delegate< TRetVal(TParams...)>::Bind (
            TClass & object) [noexcept]
```

Binds the delegate to an `object` + method (member function).

6.4.3.7 IsBindedTo() [1/3]

```
template<typename TRetVal, typename... TParams>
template<TRetVal(*)(TParams...) TFunction>
bool bricxx::Delegate< TRetVal(TParams...)>::IsBindedTo () const [constexpr], [noexcept]
```

Test if the delegate is binded to the given (non-member) function.

6.4.3.8 IsBindedTo() [2/3]

```
template<typename TRetVal, typename... TParams>
template<class TFunctor, typename = typename ::std::enable_if< ::std::is_class<TFunctor>::↩
value >::type>
bool bricxx::Delegate< TRetVal(TParams...)>::IsBindedTo (
            TFunctor & functor) const [noexcept]
```

Test if the delegate is binded to the given `functor` / lambda (`TFunctor` is auto-deduced).

6.4.3.9 IsBindedTo() [3/3]

```
template<typename TRetVal, typename... TParams>
template<class TClass, TRetVal(TClass::*)(TParams...) TMethod>
bool bricxx::Delegate< TRetVal(TParams...)>::IsBindedTo (
            TClass & object) const [noexcept]
```

Test if the delegate is binded to the given `object` + method (member function).

6.4.3.10 operator()()

```
template<typename TRetVal, typename... TParams>
template<typename... UParams>
TRetVal bricxx::Delegate< TRetVal(TParams...)>::operator() (
            UParams &&... params)
```

Invokes the stub function with the given parameters.

Remarks

Perfect forwarding and converting parameter pack to stub function (`UParams` is a deduced type, `TParams` is not).

Exceptions

BadDelegateCall	if the delegate is not binded.

The documentation for this class was generated from the following file:

- delegate.hpp

6.5 bricxx::DelegateBase< TRetVal, TParams > Class Template Reference

Base class for `Delegate` partial specializations.

```
#include <delegate.hpp>
```

Public Member Functions

- constexpr bool IsBinded () const noexcept
- constexpr operator bool () const noexcept

  Same as `IsBinded()`.
- constexpr void Unbind () noexcept
- constexpr void Reset () noexcept

  Same as `Unbind()`.

Protected Types

- using ErasedObjectType = void
- using StubFnPtr = TRetVal (∗)(ErasedObjectType∗ pTypeErasedObject, TParams... params)

Protected Member Functions

- ∼DelegateBase () noexcept
- constexpr DelegateBase & operator= (const DelegateBase &other) noexcept

Constructors.

- constexpr DelegateBase () noexcept
  Creates a delegate that is not binded.
- constexpr DelegateBase (const DelegateBase &other) noexcept

Equality operators.

- constexpr bool operator== (const DelegateBase &other) const noexcept
- constexpr bool operator!= (const DelegateBase &other) const noexcept

Static Protected Member Functions

- static TRetVal NullStub (ErasedObjectType ∗pTypeErasedObject, TParams... params)

Protected Attributes

- StubFnPtr pStub_ = &NullStub
- ErasedObjectType ∗ pTypeErasedObject_ = nullptr

6.5.1 Detailed Description

template<typename TRetVal, typename... TParams>
class bricxx::DelegateBase< TRetVal, TParams >

Base class for `Delegate` partial specializations.

Definition at line 168 of file delegate.hpp.

6.5.2 Member Typedef Documentation

6.5.2.1 ErasedObjectType

```
template<typename TRetVal, typename...  TParams>
using bricxx::DelegateBase< TRetVal, TParams >::ErasedObjectType = void  [protected]
```

Definition at line 176 of file delegate.hpp.

6.5.2.2 StubFnPtr

```
template<typename TRetVal, typename...  TParams>
using bricxx::DelegateBase< TRetVal, TParams >::StubFnPtr = TRetVal (*)(ErasedObjectType* p↩
TypeErasedObject, TParams...  params)  [protected]
```

Definition at line 177 of file delegate.hpp.

6.5.3 Constructor & Destructor Documentation

6.5.3.1 DelegateBase() [1/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::DelegateBase< TRetVal, TParams >::DelegateBase ()  [constexpr], [protected], [noexcept]
```

Creates a delegate that is not binded.

6.5.3.2 DelegateBase() [2/2]

```
template<typename TRetVal, typename...  TParams>
bricxx::DelegateBase< TRetVal, TParams >::DelegateBase (
            const DelegateBase< TRetVal, TParams > & other)  [constexpr], [protected], [noexcept]
```

6.5.3.3 ~DelegateBase()

```
template<typename TRetVal, typename...  TParams>
bricxx::DelegateBase< TRetVal, TParams >::~DelegateBase ()  [protected], [noexcept]
```

Remarks

Not virtual.

6.5.4 Member Function Documentation

6.5.4.1 IsBinded()

```
template<typename TRetVal, typename...  TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::IsBinded () const  [constexpr], [noexcept]
```

6.5.4.2 operator bool()

```
template<typename TRetVal, typename...  TParams>
bricxx::DelegateBase< TRetVal, TParams >::operator bool () const  [explicit], [constexpr],
[noexcept]
```

Same as `IsBinded()`.

6.5.4.3 Unbind()

```
template<typename TRetVal, typename... TParams>
void bricxx::DelegateBase< TRetVal, TParams >::Unbind () [constexpr], [noexcept]
```

6.5.4.4 Reset()

```
template<typename TRetVal, typename... TParams>
void bricxx::DelegateBase< TRetVal, TParams >::Reset () [constexpr], [noexcept]
```

Same as `Unbind()`.

6.5.4.5 operator=()

```
template<typename TRetVal, typename... TParams>
DelegateBase & bricxx::DelegateBase< TRetVal, TParams >::operator= (
            const DelegateBase< TRetVal, TParams > & other) [constexpr], [protected], [noexcept]
```

6.5.4.6 operator==()

```
template<typename TRetVal, typename... TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::operator== (
            const DelegateBase< TRetVal, TParams > & other) const [constexpr], [protected],
[noexcept]
```

6.5.4.7 operator"!=()

```
template<typename TRetVal, typename... TParams>
bool bricxx::DelegateBase< TRetVal, TParams >::operator!= (
            const DelegateBase< TRetVal, TParams > & other) const [constexpr], [protected],
[noexcept]
```

6.5.4.8 NullStub()

```
template<typename TRetVal, typename... TParams>
TRetVal bricxx::DelegateBase< TRetVal, TParams >::NullStub (
            ErasedObjectType * pTypeErasedObject,
            TParams... params) [static], [protected]
```

Exceptions

BadDelegateCall	This stub is used when the delegate is not binded to a "real" target.

6.5.5 Member Data Documentation

6.5.5.1 pStub_

```
template<typename TRetVal, typename...  TParams>
StubFnPtr bricxx::DelegateBase< TRetVal, TParams >::pStub_ = &NullStub  [protected]
```

Definition at line 202 of file delegate.hpp.

6.5.5.2 pTypeErasedObject_

```
template<typename TRetVal, typename...  TParams>
ErasedObjectType* bricxx::DelegateBase< TRetVal, TParams >::pTypeErasedObject_ = nullptr  [protected]
```

Definition at line 203 of file delegate.hpp.

The documentation for this class was generated from the following file:

- delegate.hpp

Chapter 7

File Documentation

7.1 delegate.hpp File Reference

```
#include <exception>
#include <type_traits>
#include <utility>
#include <memory>
```

Classes

- class bricxx::BadDelegateCall

  Exception thrown by `Delegate::operator()()` if the delegate is not binded.
- class bricxx::DelegateBase< TRetVal, TParams >

  Base class for `Delegate` partial specializations.
- class bricxx::Delegate< TRetVal(TParams...)>

  `Delegate` partial specialization for non-const.
- class bricxx::Delegate< TRetVal(TParams...) const >

  `Delegate` partial specialization for const.

Namespaces

- namespace bricxx

Macros

- #define BRICXX_DELEGATE_API

7.1.1 Macro Definition Documentation

7.1.1.1 BRICXX_DELEGATE_API

```
#define BRICXX_DELEGATE_API
```

Definition at line 58 of file delegate.hpp.

7.2 delegate.hpp

Go to the documentation of this file.

```
// MIT License: https://spdx.org/licenses/MIT.html
//
// Copyright (c) 2026 Cristian ANITA <cristian_anita@yahoo.com>
//
// Permission is hereby granted, free of charge, to any person obtaining a copy
// of this software and associated documentation files (the "Software"), to deal
// in the Software without restriction, including without limitation the rights
// to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
// copies of the Software, and to permit persons to whom the Software is
// furnished to do so, subject to the following conditions:
//
// The above copyright notice and this permission notice shall be included in all
// copies or substantial portions of the Software.
//
// THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
// EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
// MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
// IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,
// DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR
// OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE
// OR OTHER DEALINGS IN THE SOFTWARE.


#ifndef BRICXX_DELEGATE_HPP_INCLUDED_
#define BRICXX_DELEGATE_HPP_INCLUDED_


// /Zc:__cplusplus; please see:
// - MSVC now correctly reports __cplusplus: https://devblogs.microsoft.com/cppblog/msvc-now-correctly-reports-__cplusplus
// - /Zc:__cplusplus (Enable updated __cplusplus macro): https://learn.microsoft.com/en-us/cpp/build/reference/zc-cplusplus
#ifdef _MSC_VER
#   if (_MSC_VER < 1914)
#       error "msvc >= 2017 v15.7 required!!! (for /Zc:__cplusplus)"
#   endif
#endif

#if (__cplusplus < 201402L)
#   error "C++ >= 14 required!!!"
#endif


#ifdef _MSC_VER
#pragma warning(push)
// C4646: function declared with __declspec(noreturn) has non-void return type.
// DelegateBase::NullStub() has (possibly non-void) return type (it must conform to stub signature)
// but it actually always throws BadDelegateCall.
#pragma warning(disable: 4646)
#endif


#include <exception>
#include <type_traits>
#include <utility>
#include <memory>


// TODO: If Delegate is used in a shared library define BRICXX_DELEGATE_API to that library export/import macro.
#define BRICXX_DELEGATE_API


namespace bricxx {


//*****************************************************************************************************************
template <typename TSignature>
class Delegate;




//*****************************************************************************************************************
class BRICXX_DELEGATE_API BadDelegateCall final: public ::std::exception {
public:
    const char* what() const noexcept override;
};


```

```
//*****************************************************************************************************************
template <typename TRetVal, typename... TParams>
class DelegateBase {
public:
    constexpr bool IsBinded() const noexcept;
    explicit constexpr operator bool() const noexcept;

    constexpr void Unbind() noexcept;
    constexpr void Reset() noexcept;
protected:
    using ErasedObjectType = void;
    using StubFnPtr = TRetVal (*)(ErasedObjectType* pTypeErasedObject, TParams... params);


    constexpr DelegateBase() noexcept;
    constexpr DelegateBase(const DelegateBase& other) noexcept;

    ~DelegateBase() noexcept;

    constexpr DelegateBase& operator=(const DelegateBase& other) noexcept;


    constexpr bool operator==(const DelegateBase& other) const noexcept;
    constexpr bool operator!=(const DelegateBase& other) const noexcept;


    [[noreturn]] static TRetVal NullStub(ErasedObjectType* pTypeErasedObject, TParams... params);


    StubFnPtr pStub_ = &NullStub;
    ErasedObjectType* pTypeErasedObject_ = nullptr;
};




//*****************************************************************************************************************
template <typename TRetVal, typename... TParams>
class Delegate<TRetVal (TParams...)> final: private DelegateBase<TRetVal, TParams...> {
public:
    using DelegateBase<TRetVal, TParams...>::IsBinded;
    using DelegateBase<TRetVal, TParams...>::operator bool;

    using DelegateBase<TRetVal, TParams...>::Unbind;
    using DelegateBase<TRetVal, TParams...>::Reset;


    constexpr Delegate() noexcept;
    constexpr Delegate(const Delegate& other) noexcept;

    ~Delegate() noexcept;

    constexpr Delegate& operator=(const Delegate& other) noexcept;


    constexpr bool operator==(const Delegate& other) const noexcept;
    constexpr bool operator!=(const Delegate& other) const noexcept;



    template <TRetVal (*TFunction)(TParams...)>
    constexpr void Bind() noexcept;

    template <class TFunctor,
        typename = typename ::std::enable_if<
            ::std::is_class<TFunctor>::value
        >::type
    >
    void Bind(TFunctor& functor) noexcept;

    template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
    void Bind(TClass& object) noexcept;




    template <TRetVal (*TFunction)(TParams...)>
    constexpr bool IsBindedTo() const noexcept;

    template <class TFunctor,
        typename = typename ::std::enable_if<
            ::std::is_class<TFunctor>::value
        >::type
```

```
    >
    bool IsBindedTo(TFunctor& functor) const noexcept;

    template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
    bool IsBindedTo(TClass& object) const noexcept;



    template <typename... UParams>
    TRetVal operator()(UParams&&... params);
private:
    using typename DelegateBase<TRetVal, TParams...>::ErasedObjectType;
    using typename DelegateBase<TRetVal, TParams...>::StubFnPtr;



    template <TRetVal (*TFunction)(TParams...)>
    static TRetVal FunctionStub(ErasedObjectType* pTypeErasedObject, TParams... /* params */);

    template <class TFunctor>
    static TRetVal FunctorStub(ErasedObjectType* pTypeErasedObject, TParams... params);

    template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
    static TRetVal MethodStub(ErasedObjectType* pTypeErasedObject, TParams... params);

};




    //*****************************************************************************************************************
template <typename TRetVal, typename... TParams>
class Delegate<TRetVal (TParams...) const> final: private DelegateBase<TRetVal, TParams...> {
public:
    using DelegateBase<TRetVal, TParams...>::IsBinded;
    using DelegateBase<TRetVal, TParams...>::operator bool;

    using DelegateBase<TRetVal, TParams...>::Unbind;
    using DelegateBase<TRetVal, TParams...>::Reset;


    constexpr Delegate() noexcept;
    constexpr Delegate(const Delegate& other) noexcept;

    ~Delegate() noexcept;

    constexpr Delegate& operator=(const Delegate& other) noexcept;


    constexpr bool operator==(const Delegate& other) const noexcept;
    constexpr bool operator!=(const Delegate& other) const noexcept;



    template <class TFunctor,
        typename = typename ::std::enable_if<
            ::std::is_class<TFunctor>::value
        >::type
    >
    void Bind(const TFunctor& functor) noexcept;

    template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
    void Bind(const TClass& object) noexcept;



    template <class TFunctor,
        typename = typename ::std::enable_if<
            ::std::is_class<TFunctor>::value
        >::type
    >
    bool IsBindedTo(const TFunctor& functor) const noexcept;

    template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
    bool IsBindedTo(const TClass& object) const noexcept;



    template <typename... UParams>
    TRetVal operator()(UParams&&... params) const;
private:
    using typename DelegateBase<TRetVal, TParams...>::ErasedObjectType;
    using typename DelegateBase<TRetVal, TParams...>::StubFnPtr;


```

```
    template <class TFunctor>
    static TRetVal FunctorStub(ErasedObjectType* pTypeErasedObject, TParams... params);

    template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
    static TRetVal MethodStub(ErasedObjectType* pTypeErasedObject, TParams... params);

};
```

```
//*****************************************************************************************************************
// Inline functions / template implementation

inline const char* BadDelegateCall::what() const noexcept {
    return "Bad delegate call";
}
```

```
template <typename TRetVal, typename... TParams>
inline constexpr bool DelegateBase<TRetVal, TParams...>::IsBinded() const noexcept {
    return pStub_ != &NullStub;
}

template <typename TRetVal, typename... TParams>
inline constexpr DelegateBase<TRetVal, TParams...>::operator bool() const noexcept {
    return IsBinded();
}

template <typename TRetVal, typename... TParams>
inline constexpr void DelegateBase<TRetVal, TParams...>::Unbind() noexcept {
    pStub_ = &NullStub;
    pTypeErasedObject_ = nullptr;
}

template <typename TRetVal, typename... TParams>
inline constexpr void DelegateBase<TRetVal, TParams...>::Reset() noexcept {
    Unbind();
}

template <typename TRetVal, typename... TParams>
inline constexpr DelegateBase<TRetVal, TParams...>::DelegateBase() noexcept = default;

template <typename TRetVal, typename... TParams>
inline constexpr DelegateBase<TRetVal, TParams...>::DelegateBase(const DelegateBase& other) noexcept = default;

template <typename TRetVal, typename... TParams>
inline DelegateBase<TRetVal, TParams...>::~DelegateBase() noexcept = default;

template <typename TRetVal, typename... TParams>
inline constexpr DelegateBase<TRetVal, TParams...>& DelegateBase<TRetVal, TParams...>::operator=(
        const DelegateBase& other) noexcept = default;

template <typename TRetVal, typename... TParams>
inline constexpr bool DelegateBase<TRetVal, TParams...>::operator==(const DelegateBase& other) const noexcept {
    return (pStub_ == other.pStub_) && (pTypeErasedObject_ == other.pTypeErasedObject_);
}

template <typename TRetVal, typename... TParams>
inline constexpr bool DelegateBase<TRetVal, TParams...>::operator!=(const DelegateBase& other) const noexcept {
    return !(*this == other);
}

template <typename TRetVal, typename... TParams>
TRetVal DelegateBase<TRetVal, TParams...>::NullStub(ErasedObjectType* pTypeErasedObject, TParams... /* params */) {
    (void)pTypeErasedObject;

    throw BadDelegateCall();
}
```

```


template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...)>::Delegate() noexcept = default;

template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...)>::Delegate(const Delegate& other) noexcept = default;


template <typename TRetVal, typename... TParams>
inline Delegate<TRetVal (TParams...)>::~Delegate() noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...)>& Delegate<TRetVal (TParams...)>::operator=(const Delegate& other)
        noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...)>::operator==(const Delegate& other) const noexcept {
    return DelegateBase<TRetVal, TParams...>::operator==(other);
}

template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...)>::operator!=(const Delegate& other) const noexcept {
    return !(*this == other);
}


template <typename TRetVal, typename... TParams>
template <TRetVal (*TFunction)(TParams...)>
inline constexpr void Delegate<TRetVal (TParams...)>::Bind() noexcept {
    this->pStub_ = &FunctionStub<TFunction>;
    this->pTypeErasedObject_ = nullptr;
}

template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline void Delegate<TRetVal (TParams...)>::Bind(TFunctor& functor) noexcept {
    this->pStub_ = &FunctorStub<TFunctor>;
    this->pTypeErasedObject_ = ::std::addressof(functor);
}

template <typename TRetVal, typename... TParams>
template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
inline void Delegate<TRetVal (TParams...)>::Bind(TClass& object) noexcept {
    static_assert(sizeof(TClass*) == sizeof(ErasedObjectType*), "pointers size mismatch");

    this->pStub_ = &MethodStub<TClass, TMethod>;
    this->pTypeErasedObject_ = ::std::addressof(object);
}


template <typename TRetVal, typename... TParams>
template <TRetVal (*TFunction)(TParams...)>
inline constexpr bool Delegate<TRetVal (TParams...)>::IsBindedTo() const noexcept {
    Delegate delegate;
    delegate.Bind<TFunction>();
    return *this == delegate;
}

template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline bool Delegate<TRetVal (TParams...)>::IsBindedTo(TFunctor& functor) const noexcept {
    Delegate delegate;
    delegate.Bind(functor);
    return *this == delegate;
}

template <typename TRetVal, typename... TParams>
template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
inline bool Delegate<TRetVal (TParams...)>::IsBindedTo(TClass& object) const noexcept {
    Delegate delegate;
    delegate.Bind<TClass, TMethod>(object);
    return *this == delegate;
}


template <typename TRetVal, typename... TParams>
template <typename... UParams>
inline TRetVal Delegate<TRetVal (TParams...)>::operator()(UParams&&... params) {
    return (*this->pStub_)(this->pTypeErasedObject_, ::std::forward<UParams>(params)...);
}

```

```

template <typename TRetVal, typename... TParams>
template <TRetVal (*TFunction)(TParams...)>
TRetVal Delegate<TRetVal (TParams...)>::FunctionStub(ErasedObjectType* pTypeErasedObject, TParams... params) {
    (void)pTypeErasedObject;

    return (*TFunction)(::std::forward<TParams>(params)...);
}

template <typename TRetVal, typename... TParams>
template <class TFunctor>
TRetVal Delegate<TRetVal (TParams...)>::FunctorStub(ErasedObjectType* pTypeErasedObject, TParams... params) {
    return (*static_cast<TFunctor*>(pTypeErasedObject))(::std::forward<TParams>(params)...);
}

template <typename TRetVal, typename... TParams>
template <class TClass, TRetVal (TClass::*TMethod)(TParams...)>
TRetVal Delegate<TRetVal (TParams...)>::MethodStub(ErasedObjectType* pTypeErasedObject, TParams... params) {
    return (static_cast<TClass*>(pTypeErasedObject)->*TMethod)(::std::forward<TParams>(params)...);
}




template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) const>::Delegate() noexcept = default;

template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) const>::Delegate(const Delegate& other) noexcept = default;


template <typename TRetVal, typename... TParams>
inline Delegate<TRetVal (TParams...) const>::~Delegate() noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr Delegate<TRetVal (TParams...) const>& Delegate<TRetVal (TParams...) const>::operator=(const Delegate& other)
        noexcept = default;


template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...) const>::operator==(const Delegate& other) const noexcept {
    return DelegateBase<TRetVal, TParams...>::operator==(other);
}

template <typename TRetVal, typename... TParams>
inline constexpr bool Delegate<TRetVal (TParams...) const>::operator!=(const Delegate& other) const noexcept {
    return !(*this == other);
}


template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline void Delegate<TRetVal (TParams...) const>::Bind(const TFunctor& functor) noexcept {
    this->pStub_ = &FunctorStub<TFunctor>;

    // removed const will be restored in FunctorStub()
    this->pTypeErasedObject_ = const_cast<ErasedObjectType*>(static_cast<const ErasedObjectType*>(::std::addressof(functor)));
}

template <typename TRetVal, typename... TParams>
template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
inline void Delegate<TRetVal (TParams...) const>::Bind(const TClass& object) noexcept {
    static_assert(sizeof(TClass*) == sizeof(ErasedObjectType*), "pointers size mismatch");

    this->pStub_ = &MethodStub<TClass, TMethod>;

    // removed const will be restored in MethodStub()
    this->pTypeErasedObject_ = const_cast<ErasedObjectType*>(static_cast<const ErasedObjectType*>(::std::addressof(object)));
}


template <typename TRetVal, typename... TParams>
template <class TFunctor, typename>
inline bool Delegate<TRetVal (TParams...) const>::IsBindedTo(const TFunctor& functor) const noexcept {
    Delegate delegate;
    delegate.Bind(functor);
    return *this == delegate;
```

```
}

template <typename TRetVal, typename... TParams>
template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
inline bool Delegate<TRetVal (TParams...) const>::IsBindedTo(const TClass& object) const noexcept {
    Delegate delegate;
    delegate.Bind<TClass, TMethod>(object);
    return *this == delegate;
}


template <typename TRetVal, typename... TParams>
template <typename... UParams>
inline TRetVal Delegate<TRetVal (TParams...) const>::operator()(UParams&&... params) const {
    return (*this->pStub_)(this->pTypeErasedObject_, ::std::forward<UParams>(params)...);
}


template <typename TRetVal, typename... TParams>
template <class TFunctor>
TRetVal Delegate<TRetVal (TParams...) const>::FunctorStub(ErasedObjectType* pTypeErasedObject,
TParams... params) {
    // restoring removed const
    return (*static_cast<const TFunctor*>(pTypeErasedObject))(::std::forward<TParams>(params)...);
}

template <typename TRetVal, typename... TParams>
template <class TClass, TRetVal (TClass::*TMethod)(TParams...) const>
TRetVal Delegate<TRetVal (TParams...) const>::MethodStub(ErasedObjectType* pTypeErasedObject,
TParams... params) {
    // restoring removed const
    return (static_cast<const
TClass*>(pTypeErasedObject)->*TMethod)(::std::forward<TParams>(params)...);
}




}   // namespace bricxx


#ifdef _MSC_VER
#pragma warning(pop)
#endif


#endif  // BRICXX_DELEGATE_HPP_INCLUDED_
```

Index

~Delegate
- bricxx::Delegate< TRetVal(TParams...) const >, 15
- bricxx::Delegate< TRetVal(TParams...)>, 18

~DelegateBase
- bricxx::DelegateBase< TRetVal, TParams >, 22

Bind
- bricxx::Delegate< TRetVal(TParams...) const >, 15, 16
- bricxx::Delegate< TRetVal(TParams...)>, 19

bricxx, 9

bricxx::BadDelegateCall, 11
- what, 11

bricxx::Delegate< TRetVal(TParams...) const >, 13
- ~Delegate, 15
- Bind, 15, 16
- Delegate, 15
- IsBindedTo, 16
- operator!=, 15
- operator(), 16
- operator=, 15
- operator==, 15

bricxx::Delegate< TRetVal(TParams...)>, 17
- ~Delegate, 18
- Bind, 19
- Delegate, 18
- IsBindedTo, 19
- operator!=, 18
- operator(), 20
- operator=, 18
- operator==, 18

bricxx::Delegate< TSignature >, 12

bricxx::DelegateBase< TRetVal, TParams >, 20
- ~DelegateBase, 22
- DelegateBase, 22
- ErasedObjectType, 21
- IsBinded, 22
- NullStub, 23
- operator bool, 22
- operator!=, 23
- operator=, 23
- operator==, 23
- pStub_, 24
- pTypeErasedObject_, 24
- Reset, 23
- StubFnPtr, 21
- Unbind, 22

BRICXX_DELEGATE_API
- delegate.hpp, 25

Delegate
- bricxx::Delegate< TRetVal(TParams...) const >, 15
- bricxx::Delegate< TRetVal(TParams...)>, 18

delegate.hpp, 25
- BRICXX_DELEGATE_API, 25

DelegateBase
- bricxx::DelegateBase< TRetVal, TParams >, 22

ErasedObjectType
- bricxx::DelegateBase< TRetVal, TParams >, 21

IsBinded
- bricxx::DelegateBase< TRetVal, TParams >, 22

IsBindedTo
- bricxx::Delegate< TRetVal(TParams...) const >, 16
- bricxx::Delegate< TRetVal(TParams...)>, 19

NullStub
- bricxx::DelegateBase< TRetVal, TParams >, 23

operator bool
- bricxx::DelegateBase< TRetVal, TParams >, 22

operator!=
- bricxx::Delegate< TRetVal(TParams...) const >, 15
- bricxx::Delegate< TRetVal(TParams...)>, 18
- bricxx::DelegateBase< TRetVal, TParams >, 23

operator()
- bricxx::Delegate< TRetVal(TParams...) const >, 16
- bricxx::Delegate< TRetVal(TParams...)>, 20

operator=
- bricxx::Delegate< TRetVal(TParams...) const >, 15
- bricxx::Delegate< TRetVal(TParams...)>, 18
- bricxx::DelegateBase< TRetVal, TParams >, 23

operator==
- bricxx::Delegate< TRetVal(TParams...) const >, 15
- bricxx::Delegate< TRetVal(TParams...)>, 18
- bricxx::DelegateBase< TRetVal, TParams >, 23

pStub_
- bricxx::DelegateBase< TRetVal, TParams >, 24

pTypeErasedObject_
- bricxx::DelegateBase< TRetVal, TParams >, 24

Reset
- bricxx::DelegateBase< TRetVal, TParams >, 23

StubFnPtr
bricxx::DelegateBase< TRetVal, TParams >, 21

Unbind
bricxx::DelegateBase< TRetVal, TParams >, 22

what
bricxx::BadDelegateCall, 11